Curs de Inteligență Artificială - 2022

Editie ingrijita de conf.univ.dr. N. Popescu-Bodorin

Curs 5-6 – Retele Neuronale

Facultatea de Informatica si Inginerie, Universitatea Spiru Haret

Titulari: Conf.univ.dr. N. Popescu-Bodorin, bodorin(a)live.com (2022)
Expert Asociat Drd. A.S. Moloiu, ale.stef.moloiu(a)gmail.com (2020, 2021)

(c) Alexandra Moloiu, 2020

Conținut Curs

Inteligența artificială noțiuni fundamentale
Învățare automată generalități
Regresie Logistică / Neuronul Artificial
Metrici de evaluare a performanței
Rețele neuronale, noțiuni generale
Perceptronul cu mai multe straturi, algoritmul de retropropagare
Algoritmi de optimizare pe gradient (SGD, RMSProp, Adam)
Erori bias versus variance
Deep Learning
Rețele Convoluționale
Arhitecturi de rețele deep
Sisteme biometrice bazate pe învățarea aprofundată. Studiu de caz verificarea facială
Mașini inteligente
Recapitulare

Motivele folosirii învățării automate

- Pentru unele probleme nu avem un algoritm
- Exemple:
  - recunoașterea fețelor, recunoașterea caracterelor, discriminare spam, non-spam
  - gruparea produselor care de obicei sunt cumpărate împreună
- De ce?
- Oamenii sunt adesea foarte buni la îndeplinirea unei sarcini în mod inconștient, dar nu sunt în măsură să explice cum. Dacă nu putem explica experiența noastră, nu putem scrie un program de calculator.
- Sarcina se poate schimba în timp sau în funcție de situație (de exemplu, definiția subiectivă a spam-ului); avem nevoie de un cadru flexibil pentru a îndeplini sarcina.
- Uneori nici măcar nu știm ce căutăm, dar am dori ca o mașină să ne ofere întrebări inteligente, precum și răspunsuri.

Date + Cunoștințe + algoritmi

- Am vrea ca algoritmul pentru aceste sarcini să fie extras automat de calculator.
- Cum?
- Colectăm un set de exemple
- Construim algoritmi capabili să însușească algoritmul legat de sarcini din date
- Ceea ce ne lipsește în cunoaștere, este compensat prin utilizarea datelor.
- Pe de altă parte, datele de la sine nu sunt suficiente: avem nevoie în continuare de cunoștințele experților pentru a proiecta sistemul de învățarea automată (Machine Learning ML)
- Cunoștințe anterioare (prior knowledge): informații despre problema disponibilă în plus față de datele de **antrenare**
- De ce?
- fără cunoștințe anterioare învățarea unui model ML din datele finite (și incomplete) este o problemă nedefinită bine (nu există o soluție unică)
- **Extragerea automată a unui algoritm din date și cunoștințe anterioare**

Cunoștințe anterioare

Listarea tuturor intrărilor este imposibilă sau inutilă:

- Până și o funcție simplă necesită un tabel cu o infinitate (nenumărabilă) de numere de input.
- OCR pe o mica imagine (32x32) ar avea nevoie de 255 **32*32

Folosind informațiile anterioare despre o problemă specifică, se poate îmbunătății performanța prin favorizarea de soluții cunoscute pentru a crește performanța în situații similare.

De examplu:

- Presupunem că nu există schimbări radicale, de obicei schimbări mici în input nu cauzează schimbări mari în output.
- Exemplu: în OCR un singur pixel e improbabil sa schimbe un 4 într-un 6
- Outputul poate fi presupus ca invariabil la unele transformări
  - schimbare de perspectivă (shear) și translatare pentru OCR,
  - pitch și viteză în recunoașterea vorbirii
- Similar R-CNN sau celelalte retele pentru segmentare semantica au arhitecturi adaptate problemei

Modele

- Un model este o reprezentare simplificată a unui element din realitate, simplificările sunt realizate eliminând detalii care nu sunt necesare, pentru a ne putea concentra pe aspectele pe care vrem sa le înțelegem.
- Aceste simplificări se bazează pe ipoteze; aceste ipoteze/presupuneri ar putea fi adevărate în anumite situații, dar ar putea să nu fie în alte situații. Aceasta implică faptul că un model care explică o anumită situație poate eșua într-o altă situație.
- În funcție de problemă este important să se evalueze compromisurile dintre viteză, acuratețe și complexitate a diferitelor modele și algoritmi și să se găsească un model care să funcționeze cel mai bine pentru acea problemă.

Presupuneri

- Credem că există un proces care explică datele pe care le observăm
- Încercăm să învățăm un model general folosind cantități de date limitate și cunoștințe anterioare.
- Chiar dacă nu observăm întregul proces, nu avem toate datele și atributele putem construi o aproximație bună.
- Modelul ar trebui sa fie capabil să facă predicții destul de corecte pentru date noi
- **Scopul este de a învăța un model simplu cu capacitate de generalizare**

Limitări

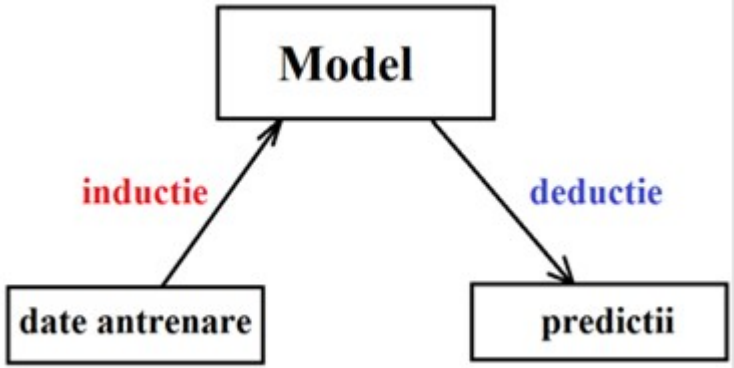
- Algoritmul ar putea să învețe ceva diferit de ceea ce ne dorim.
- Exemplu tancuri camuflate
- setul de date de antrenare a fost compus din 100 de imagini cu tancuri camuflate într-o pădure preluate într-o zi și 100 de imagini preluate a doua zi din aceeași pădure dar fără tancuri
- un algoritm a fost antrenat pentru a clasifica imaginile în două clase, cu tancuri camuflate sau fără tancuri
- Predicțiile algoritmului au fost 100% corecte (toate imaginile clasificate corect)
- Însă, imaginile cu tancuri camuflate au fost preluate într-o zi înnorată, celelalte într-o zi însorită.
- Rețeaua a învățat ceva mai simplu și total diferit, să clasifice imaginile înnorate de cele însorite.

No free lunch

- Teorema “No Free Lunch” - nicio masă gratuită
- “No Free Lunch” afirmă că nu există nici un model care să funcționeze cel mai bine pentru fiecare problemă.
- Presupunerile unui model foarte bun pentru o problemă ar putea să nu funcționeze atât de bine pentru altă problemă, așa că în învățarea automată trebuie să încercați mai multe modele și să găsiți unul care funcționează cel mai bine pentru o anumită problemă.
- Performanța în medie pe toate problemele posibile (cu o probabilitate a priori uniformă în spațiul problemelor) a tuturor clasificatorilor este aceeași.

Antrenare și predicții

- **Antrenarea** este procesul în care dându-se date și un algoritm este învațat un model.
- **Inferența** (în ML) este procesul prin care dându-se date și modelul instruit/antrenat se fac predicții.
- Antrenarea de obicei este un proces de lungă durată.
- Inferența ar trebui să se producă într-un timp insezizabil.



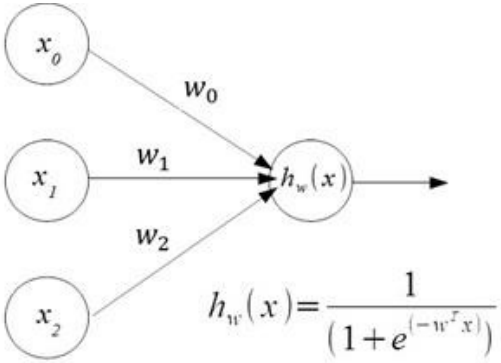
Regresie logistică

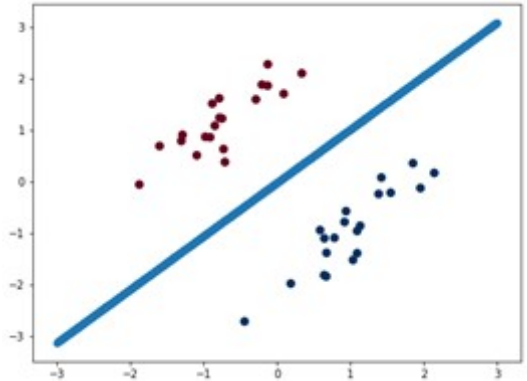
- Deși se numește regresie logistică algoritmul este folosit pentru clasificare
- $z = w_0 + w_1 * x_1 + \cdots + w_m x_m$
- $a = sigm(z) = \frac{1}{1+ e^{-z}}$
- Funcția cost:
- $L(W) = -\frac{1}{n}\sum_{i=1}^{n} y_i * \log(a_i) + (1 - y_i) * \log(1 - a_i)$

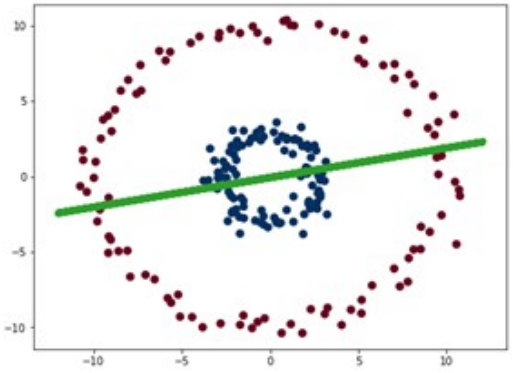
- a $>$ 0.5 - clasa 1
- a $<$ 0.5 – clasa 0

Regresie Logistică - Capacitatea unui neuron

- Regresia logistică (logistic regression) poate fi privită ca o rețea neuronală cu un singur strat cu un neuron.
- Un neuronul calculează suma ponderată a inputului și aplică o funcție de activare. În cazul regresiei logistice funcția este sigmoid.
- z = (W*X + b) a = sigm(z)
- z – suma ponderată a inputului(preactivare) a - activarea
- Ce granițe poate produce un singur neuron? Ce probleme poate rezolva?





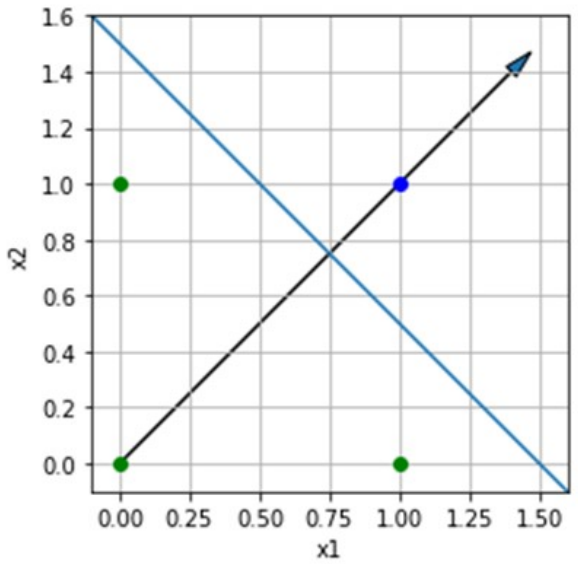


Exemplu AND

- z = -30∗1+20∗x_1+20∗x_2
- a=sigm(z)= 1/(1+e**(-z))

- Ecuația generală a dreptei
- -30 + 20*x_1 + 20*x_2 = 0

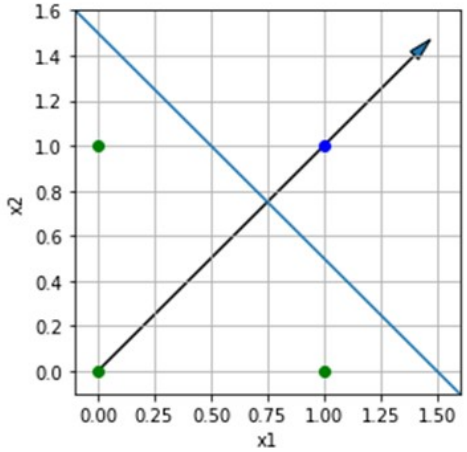
- Ecuația explicită
- -30 + 20*x_1 = -20*x_2 / *(-1/20)
- 1.5 –x_1 = x_2
- 1.5 intercept -1 panta

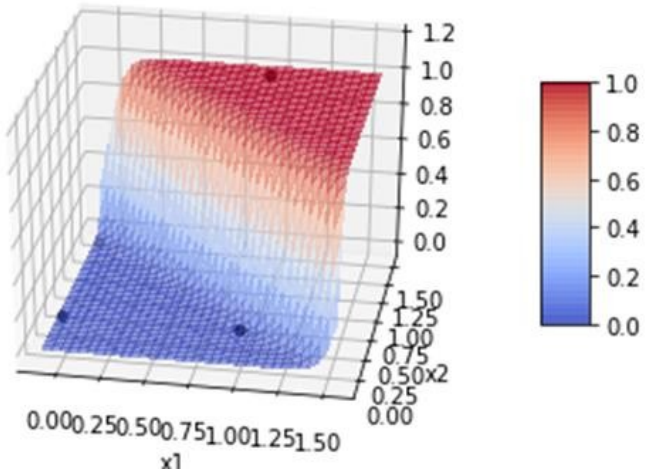


- vectorul de ponderi [20, 20] este perpendicular pe drepta de separație
- biasul 30 este proporțional cu interceptul (-30/(-1/20) = 1.5)

AND

- punctele deasupra dreptei sunt clasificate ca pozitiv (clasa 1)
- punctele de sub dreaptă ca negativ (clasa 0)
- cu cât punctele sunt mai îndepărtate de dreaptă cu atât suntem mai siguri că punctul aparține clasei respective
- dacă punctele se află exact pe dreaptă, pe granița de separație, nu știm cărei clase aparțin
- funcția sigmoid limitează outputul de la (-inf, +inf) la valori între 0 și 1

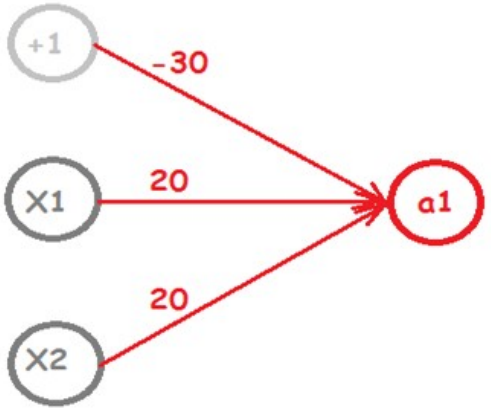




AND

- $z_1 = -30 * 1 + 20 * x_1 + 20 * x_2$
- $x_1 = 0 \quad x_2 = 0$
- $z_1 = -30 * 1 + 20 * 0 + 20 * 0 = -30$
- $a_1 = sigm(z_1) = \frac{1}{1+e^{-z_1}} \approx 0$

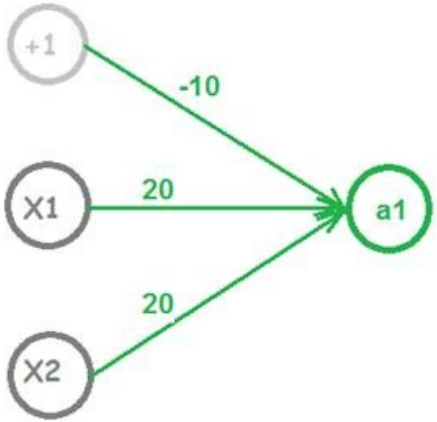
- $x_1 = 0 \quad x_2 = 1$
- $z_1 = -30 * 1 + 20 * 0 + 20 * 1 = -10$
- $a_1 = sigm(z_1) = \frac{1}{1+e^{-10}} \approx 0$
-
- $x_1 = 1 \quad x_2 = 0$
- $z_1 = -30 * 1 + 20 * 1 + 20 * 0 = -10$
- $a_1 = sigm(z_1) = \frac{1}{1+e^{-10}} \approx 0$

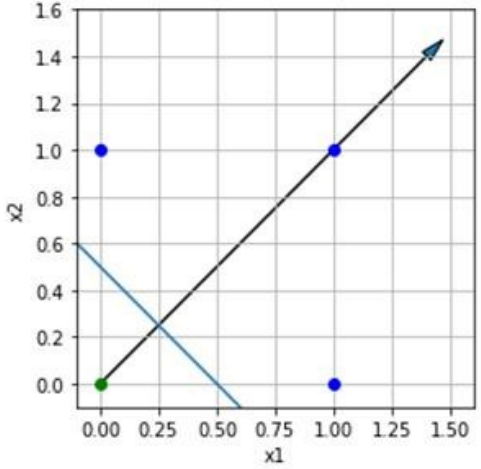


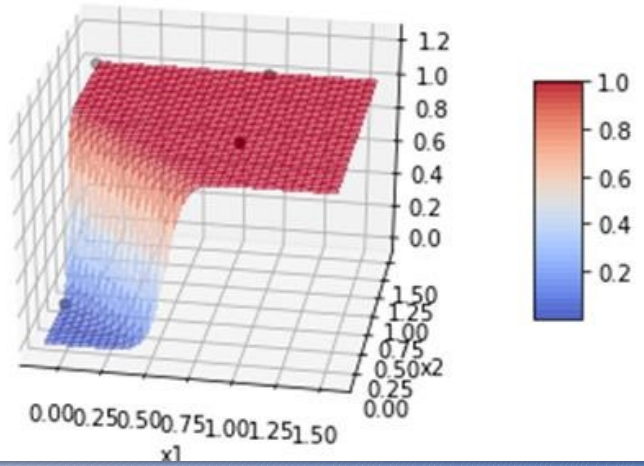
- $x_1 = 1 \quad x_2 = 1$
- $z_1 = -30 * 1 + 20 * 1 + 20 * 1 = 10$
- $a_1 = sigm(z_1) = \frac{1}{1+e^{10}} \approx 1$

OR

- z = -10 + 20*x_1 + 20*x_2
- 0.5 –x_1 = x_2 deapta de separație a claselor
- vectorul de ponderi [20, 20] este perpendicular pe drepta de separație
- biasul -10 este proporțional cu interceptul (-10/(-1/20) = 0.5)

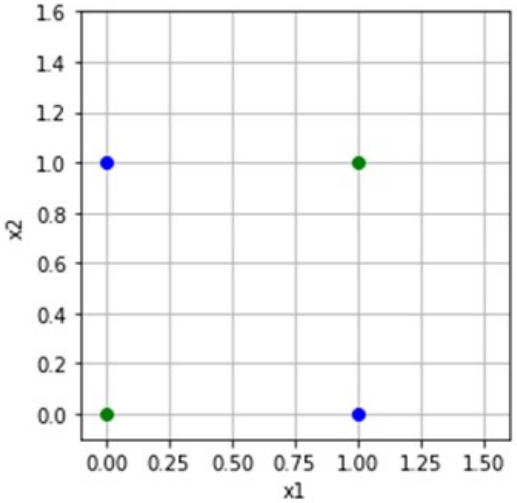






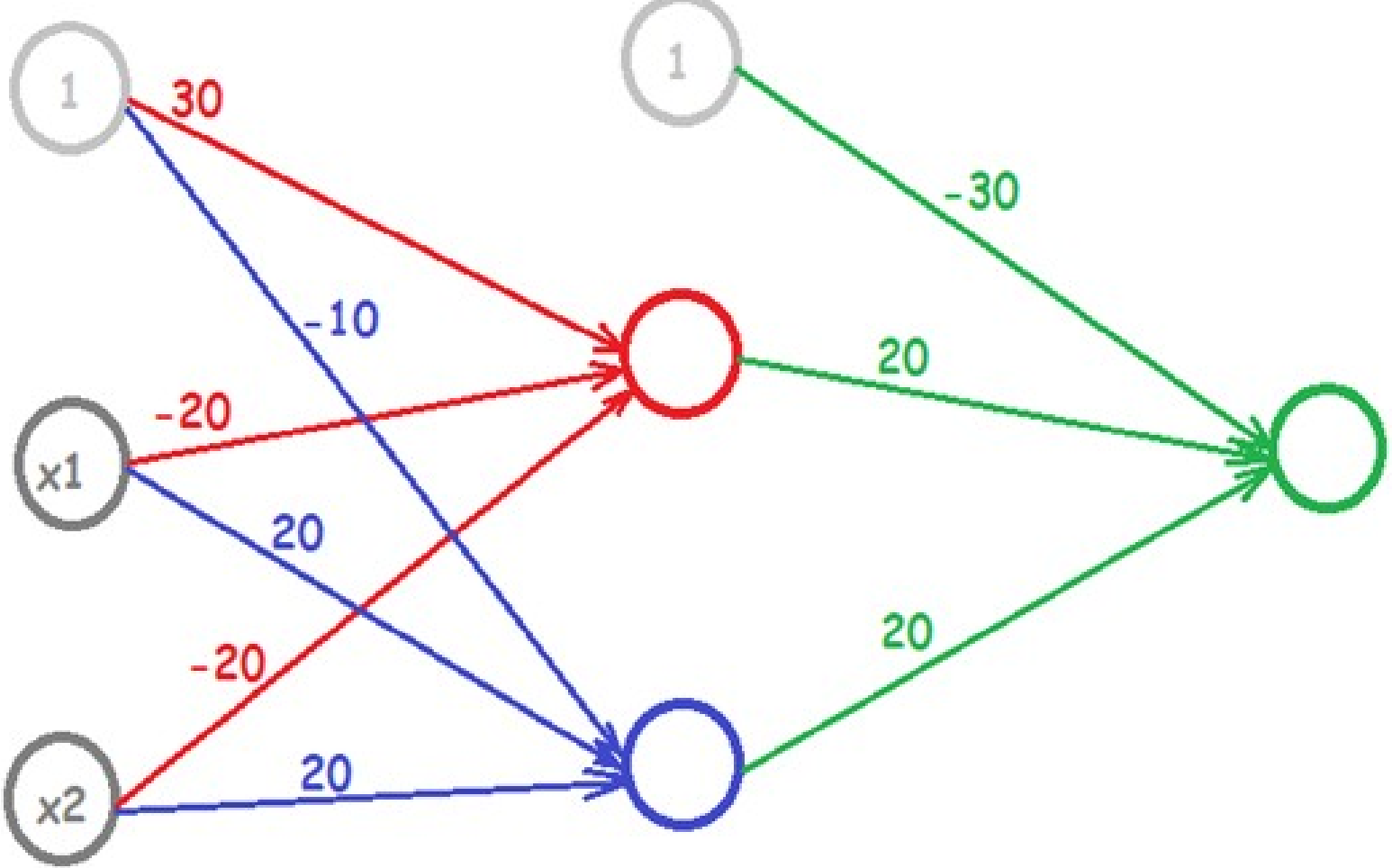
XOR (exclusive or) sau exclusiv

- Poate un singur neuron să separe bine clasele? Poate o dreaptă să delimiteze corect clasele?
- Cum ar trebui să arate granițele de separație?
- Dacă am folosi mai mulți neuroni cum ar arăta rezultatul?
- Ar fi util să folosim mai mulți neuroni intermediari?
- Ce s-ar întâmpla dacă neuronii ar avea funcții de activare liniare?
- O combinație liniară de funcții liniare este tot o funcție liniară.
- Exemplu:
- z1, z2 – primul start ascuns
- z3 – al doilea strat ascuns
- z1(x, y, z) = 2x + 3y -5z + 16
- z2(x, y, z) = 3x – 2y +6z -12
- z3(z1, z2) = 2z2 + 3z3 + 12
- z3(z1, z2) =2*(2x+3y – 5z+16) + 3*(3x – 2y +6z -12)+12= 4x+6y-10z+32+9x-6y+18z-36+12 = 13x +8z + 18
- Un singur neuron ar putea ajunge direct la output z1 = 13x +8z + 18



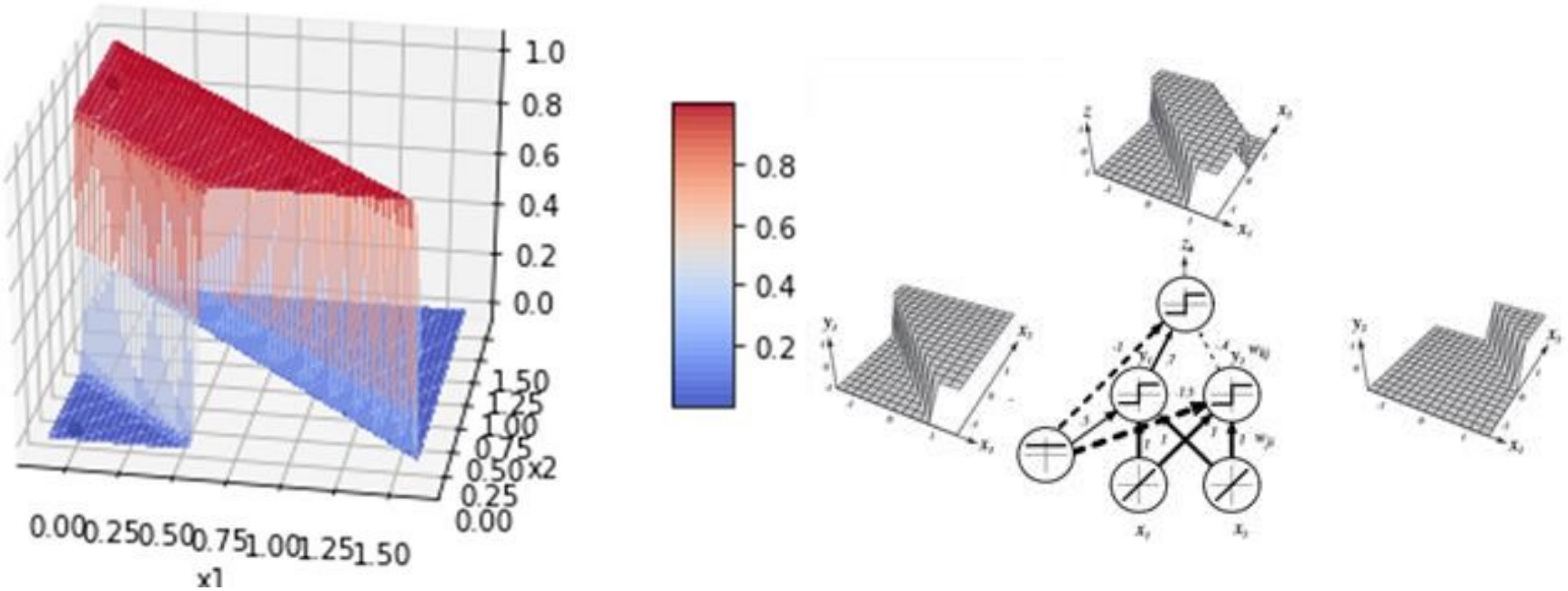
R

- c
- c
  u
- z
- a
- z
- a
- z
- a
- a
- a
- a3 = 1/(1+np.exp(-(-30 + 20*a1 + 20*a2)))



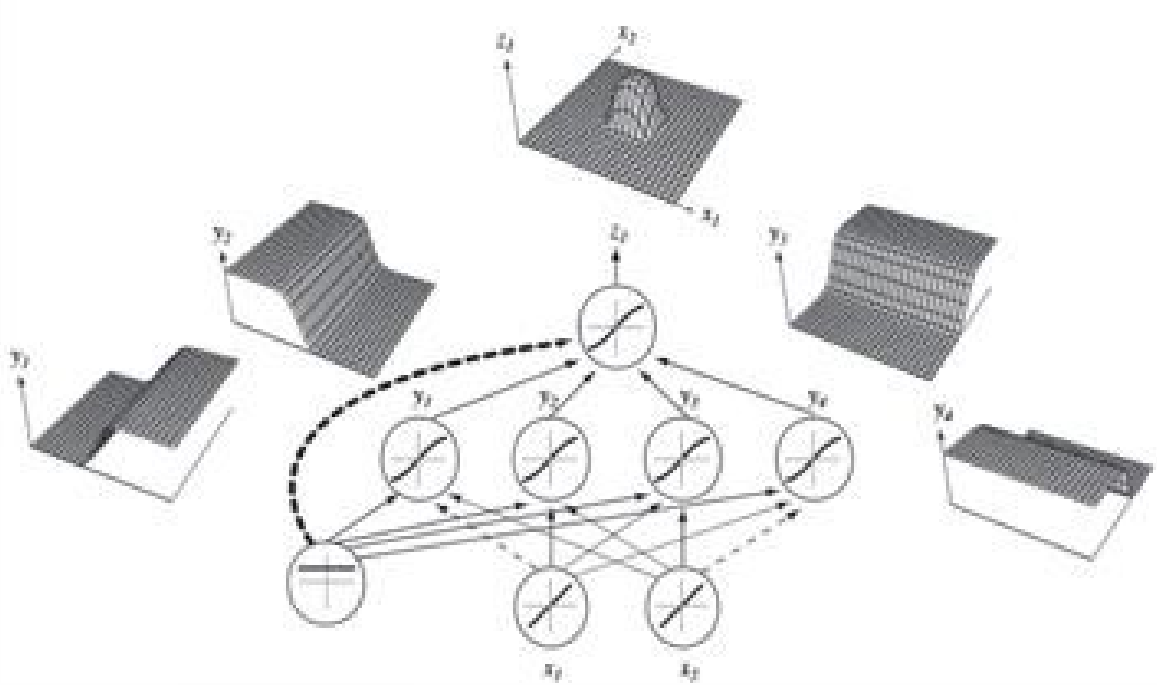
XOR - graniță

Folosind neuroni intermediari (strat ascuns, hidden layer) am obținut o graniță mai complexă:



Compunere granițe complexe

mai mulți neuroni intermediari pot genera granițe oricât de complexe.

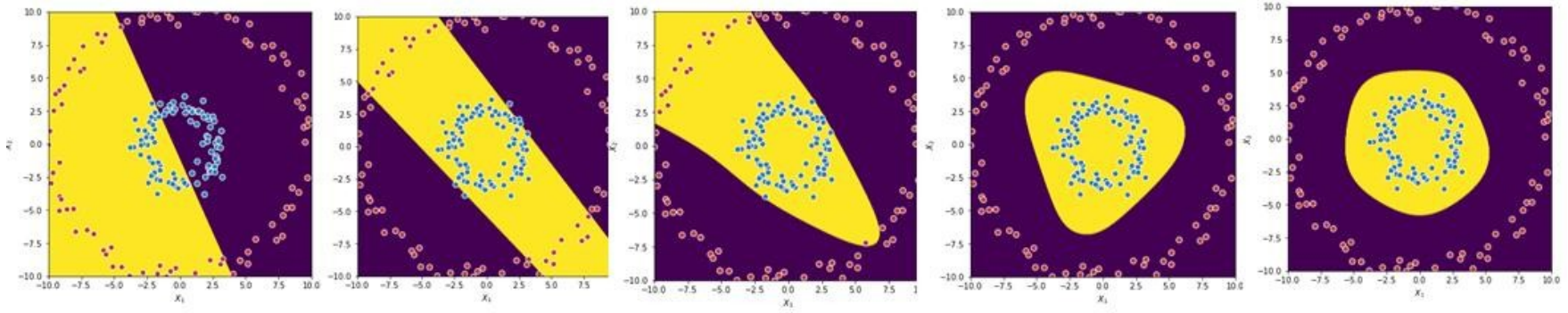


XOR

x1	x2	z1	z2	a1	a2	z	output	true output
0	0	30	-10	1	0	-10	0	0
0	1	10	10	1	1	10	1	1
1	0	10	10	1	1	10	1	1
1	1	-10	30	0	1	-10	0	0

Granițe de separație clase

- Observăm că odată cu creșterea numărului de neuroni (1, 2, 3, 6, 8, 30), modelul este capabil să clasifice punctele cât mai bine. Granița de separație este complexă datorită faptului că este o combinație neliniară (funcție activare neliniară) de granițe.
- La un nivel mai abstract am putea vedea procesul ca o combinare neliniară a mai multor clasificatori. În concluzie putem afirma faptul că un strat intermediar/hidden/ascuns cu mai mulți neuroni cu funcții de activare neliniare pot rezolva o problemă de clasificare cu date neseparabile liniar.



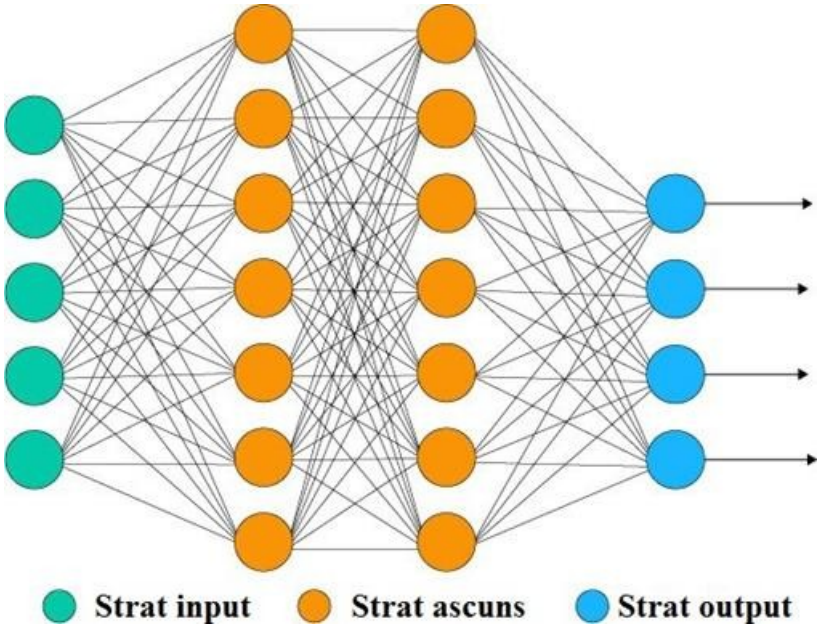
Teorema aproximatorului universal

- În cazul în care probleme sunt mai complexe este nevoie de mai mulți neuroni pe stratul ascuns (pentru a putea reprezenta/aproxima granițe complexe).
- **Un singur strat ascuns ar putea aproxima orice funcție continuă dându-se un număr suficient de neuroni.** Nu există însă nicio garanție că există algoritmul care să găsească parametrii funcției.
- **Universal approximation theorem**
- (Hornik, 1991) - ‘‘a single hidden layer neural network with a linear output unit can approximate any continuous function arbitrarily well, given enough hidden units’’
- Teorema este valabilă pentru funcții de activare neliniare precum funcția sigmoid.

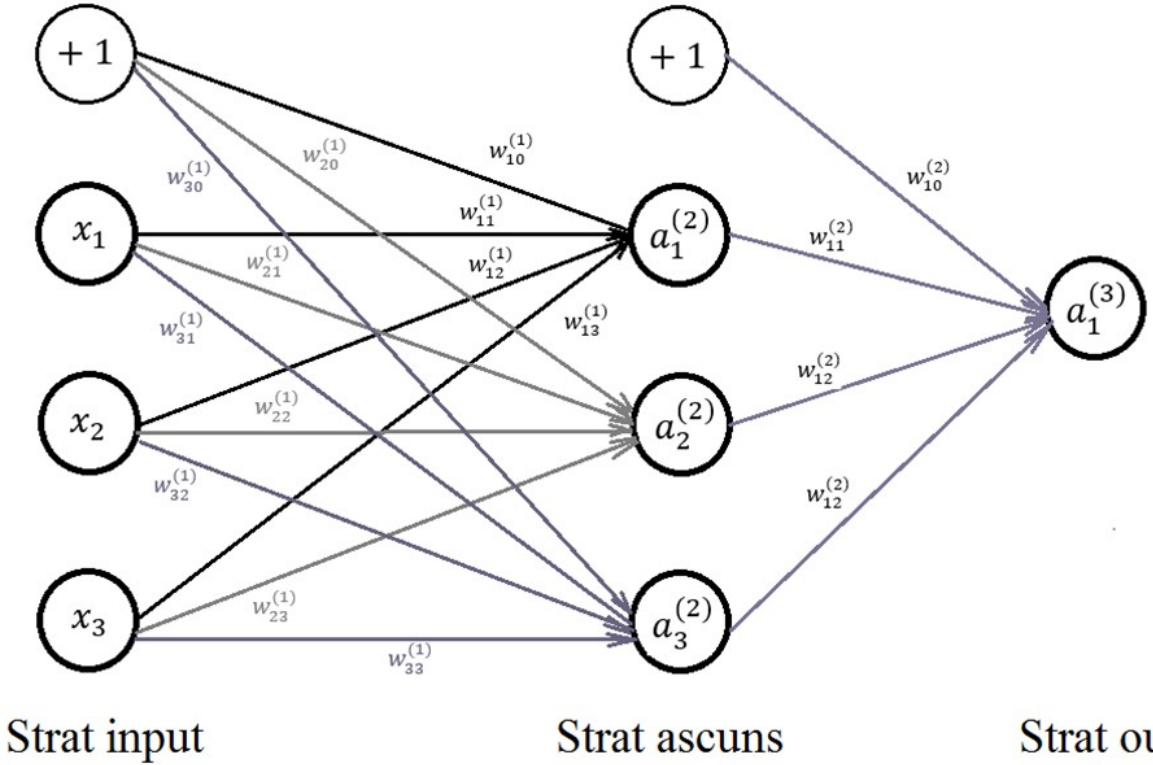
Exemplu rețea

- Rețea neuronală cu 2 straturi ascunse
- Fără bias
- 5 caracteristici ca input
- 4 clase output
- 7 neuroni primul strat ascuns
- 7 neuroni al doilea strat ascuns

- La ce folosește biasul?



Rețea neuronală



Feed Forward

- $a_0^{(2)} = 1$ – unitatea bias
- $a_1^{(2)} = g\left(z_1^{(2)}\right)$ $\quad z_1^{(2)} = w_{10}^{(1)} x_0 + w_{11}^{(1)} x_1 + w_{12}^{(1)} x_2 + w_{13}^{(1)} x_3$
- $a_2^{(2)} = g\left(z_2^{(2)}\right)$ $\quad z_2^{(2)} = w_{20}^{(1)} x_0 + w_{21}^{(1)} x_1 + w_{22}^{(1)} x_2 + w_{23}^{(1)} x_3$
- $a_3^{(2)} = g\left(z_3^{(2)}\right)$ $\quad z_3^{(2)} = w_{30}^{(1)} x_0 + w_{31}^{(1)} x_1 + w_{32}^{(1)} x_2 + w_{33}^{(1)} x_3$

- $h = a_1^{(3)} = g\left(z_1^{(3)}\right) = g(w_{10}^{(2)} a_0^{(2)} + w_{11}^{(2)} a_1^{(2)} + w_{12}^{(2)} a_2^{(2)} + w_{13}^{(2)} a_3^{(2)})$

Feed Forward

$$[x1 \quad x2 \quad x3] \begin{bmatrix} w_{11}^{(1)} \\ w_{12}^{(1)} \\ w_{13}^{(1)} \end{bmatrix} + w_{10}^{(1)} = z_1$$

$$[x1 \quad x2 \quad x3] \begin{bmatrix} w_{11}^{(1)} & w_{21}^{(1)} & w_{31}^{(1)} \\ w_{12}^{(1)} & w_{22}^{(1)} & w_{32}^{(1)} \\ w_{13}^{(1)} & w_{23}^{(1)} & w_{33}^{(1)} \end{bmatrix} + \begin{bmatrix} w_{10}^{(1)} & w_{20}^{(1)} & w_{30}^{(1)} \end{bmatrix} = [z_1 \quad z_2 \quad z_3]$$

	caracteristica1	*caracteristica2*	...	*caracteristica m*
exemplul1	$x1_1$	$x1_2$		$x1_m$
exemplul2	$x2_1$	$x2_1$		$x2_m$
...				
exemplul n	xn_1	xn_2		xn_m

Calculare număr parametrii

- Calcularea numărului de parametrii (weights) ale rețelei cu următoarea arhitecturi:
- strat input: 200 caracteristici;
- primul strat ascuns: 100 neuroni;
- al doilea strat ascuns: 50 de neuroni
- strat output: 3 neuroni (3 clase)
- Toate straturile sunt dense și folosesc bias.

- Numărul de legături între stratul de input și primul strat ascuns:
- (200 + 1)*100 = 20100
- Numărul de ponderi între primul strat ascuns și al doilea:
- (100+1)*50 = 5050
- Numărul de legături între al doilea strat ascuns și stratul de output:
- (50+1)*3 = 153
- 201*100 + 101*50 +51*3 = 25,303

Calculare număr parametrii

- Calcularea numărului de ponderi ale rețelei cu următoarea arhitecturi:
- Input: imagine 28x28
- Primul strat ascuns: 500 neuroni
- Al doilea strat ascuns: 300 neuroni
- Output: 10 clase
- Toate straturile sunt dense și folosesc bias.

- Număr parametrii:
- (28*28+1)*500 + (500+1)*300 + (300+1)*10 = 545,810

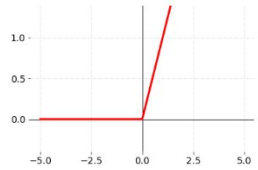
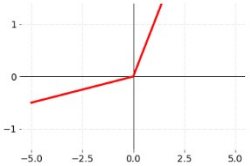
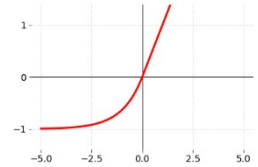
Număr straturi

- Rețelele neuronale cu mai multe straturi ascunse pot genera funcții mai complexe cu un număr mai mic de parametrii.

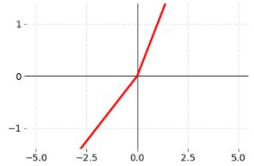
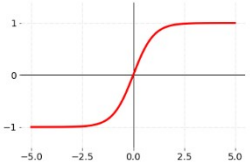
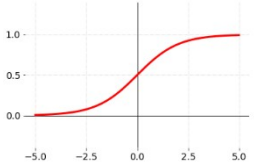
- exemplu:
- input 100 caracteristici strat hidden 1000 strat output 10 vs. două straturi hidden de 300 respectiv 100;
- 111,010 vs 61,410
- De ce ar conta numărul de parametrii? Memorie?

- **Optimizarea unui număr mare de parametrii necesită un număr mai mare de exemple de antrenare.**

Funcții de activare

ReLU		$\mathrm{ReLU}(z) = \max(0, z)$	$f'z=\{0,\ z<0 \quad 1,\ z\geq 0$
Leaky ReLU		$\mathrm{LReLU}(z)=\{\alpha z,\ z<0 \quad z,\ z\geq 0\ \}$	$\mathrm{LReLU}'z=\{\alpha, z<0 \quad 1,\ z\geq 0$
SELU		$\mathrm{SELU}z= \lambda\{\alpha e^{**}z - \alpha,\ z<0 \quad z,\ z\geq 0\ \}$	$\mathrm{SELU}'z= \lambda\{ez,\ z<0 \quad 1,\ z\geq 0$

Funcții de activare

ELU		fx={α(e**z-1), z<0 z, x≥0	f'z={f(z)+α, z<0 z, z≥0
PReLU		fx={αz, z<0 z, z≥0	f'z={α, z<0 1, z≥0
TanH		fz=tanhz=2/(1+e**-2z)– 1	f'z=1 - f(z)**2
Sigmoid		sigmoid(z)=σ=1/(1+e**-z)	f'z=f(z)(1 - f(z))

Funcții de activare

- ReLU (Rectified Linear Unit)
- Funcția este neliniară, combinațiile de ReLU sunt de asemenea neliniare. Orice funcție poate fi aproximată folosind ReLU.
- Folosirea funcției sigmoid este de preferat în cazul antrenării rețelelor cu puține straturi ascunse.
- În cazul în care avem o arhitectură mai complexă, mai multe straturi cu mai mulți neuroni este de preferat ReLu datorită faptului că introduce sparsitate, o parte din neuroni vor avea valoarea 0, nu vor fi activați.
- Dying ReLu – pentru input negativ, relu este 0 (line orizontală), gradientul este 0, ponderile neuronului nu sunt ajustate, rămân blocate la o anumită valoare. Pentru a evita problema se poate folosi leaky relu.

Funcții de activare strat output

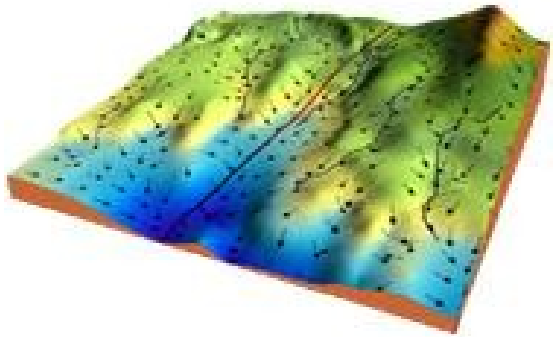
- În cazul problemelor de clasificare multiclasă, ultimul strat are un număr de neuroni egal cu numărul de clase
- În cazul clasificării binare, un singur neuron de output poate fi suficient.
- pentru a putea interpreta rezultatele ca repartiții probabilistice posterioare, probabilitatea clasei dându-se datele, p(clasă|date) este folosită ca funție de activare funcția softmax
- Aceasta este utilizată pentru a impune constrângerile rezultate din axiomele teoriei probabilităților, transpunând valorile pe intervalul [0, 1] cu suma acestora egală cu 1.
- Softmax: $\sigma(x)_j = \frac{e^{x_j}}{\sum_{c=1}^{C} e^{x_c}}$, unde C reprezintă numărul de clase

Cost

- Pentru a evalua performanța învățării în fiecare iterație este folosită o funcție cost, numită și funcție de pierdere care, calculează media energiei erorii.
- O eroare este reprezentată, în cazul unui sistem de recunoaștere de o clasificare incorectă.
- Cea mai cunoscută funcție de pierdere este entropia încrucișată (en. categorical cross entropy), echivalentă cu negativul logaritmului unei funcții parametrice a unui model statistic (en. negative log likelihood):
-
- $J(p,q) = -\sum_x p(x)\log(q(x))$,
-
- p reprezintă probabilitatea posterioară reală și q probabilitatea posterioară estimată.

Retropropagarea

- Backpropagation (retropropagare) - propagarea înapoi a erorilor - algoritmul care transferă informația despre eroare de la funcția cost la toate ponderile din interior, permițând modificarea acestora pentru minimizarea erorii.
- Eroarea în medie este calculată la ieșire și distribuită înapoi în straturile rețelei.
- Retropropagarea este folosita pentru antrenarea rețelelor neuronale.
- Scopul este actualizarea ponderilor din rețea astfel încât acestea să determine outputul să fie mai aproape de cel real, minimizând astfel eroarea pentru fiecare neuron de ieșire și rețeaua ca întreg.



Retropropagarea

- Algoritmul Backpropagation caută valoarea minimă a funcției de eroare în spațiul ponderilor utilizând o tehnică numită regulă delta sau coborârea pe gradient. Ponderile care minimizează funcția de eroare sunt considerate a fi o soluție la problema de învățare.

- Derivatele parțiale ale funcției cost reprezintă un factor de senzitivitate și determină direcția de căutare a ponderilor optime. Coborârea pe gradient este metoda cea mai folosită pentru minimizarea funcției obiectiv, în cazul de față reprezentată de funcția cost, deși nu garantează găsirea optimului global. Aceasta modifică ponderile în direcția opusă gradientului cu un pas de actualizare specificat η (hiperparametru). Derivatele parțiale sunt calculate folosind chain rule.

Optimizatori

- Actualizarea ponderilor:
- $w_n = w_n - \eta \nabla_{w_n} J(w)$
- Alegerea pasului, η, este dificilă. Dacă acesta este prea mic procesul este lent iar dacă este prea mare valorile fluctuează în jurul minimiului sau chiar diverg. Un alt neajuns al metodei este acela că pasul este identic pentru toate caracteristicile. Evitarea blocării în puncte de șa, puncte în care într-o dimensiune panta este ascendentă iar în alta descendentă, în care gradientul este aproape de 0 în toate direcțiile, și în minime locale, soluții suboptimale este problema cea mai relevantă în cazul rețelelor neuronale, funcția cost nefiind de obicei convexă.
- O altă problemă este reprezentată de suprafețele curbate în care panta este mult mai mare într-o dimensiune decât în alta. Algoritmii cei mai des utilizați în învățarea aprofundată pentru evitarea acestor probleme sunt Momentum, Gradientul accelerat Nesterov / Momentul Nesterov, Adagrad, Adadelts, RMSprop și Adam.
- Metoda momentelor accelerează coborârea pe gradient în direcția relevantă, și atenuează oscilațiile folosind o fracțiune, γ, din rezultatele anterioare, în pasul curent:
- $w_n = w_n - [\gamma(\eta \nabla_{w_{n_{t-1}}} J(w_{n_{t-1}})) + \eta \nabla_{w_n} J(w)]$
- Metoda Mementului Nesterov, comparabilă în performanță cu metodele Newton, folosește o aproximare a poziției viitoare a parametrilor pentru a putea realiza eventuale corecții:
- $w_n = w_n - [\gamma(\eta \nabla_{w_{n_{t-1}}} J(w_{n_{t-1}})) + \eta \nabla_{w_n} J\left(w - \gamma(\eta \nabla_{w_{n_{t-1}}} J(w_{n_{t-1}}))\right)]$
- Valoarea folosită pentru γ în cele mai multe probleme în care este aplicată învățarea aprofundată este 0.9.

Optimizatori

- O trecere a tuturor exemplelor de antrenare prin rețea și ajustarea ponderilor se numește o epocă.
- În funcție de datele folosite pentru ajustarea ponderilor , există mai multe variante de coborâre pe gradient.
- **Batch gradient descent**
- Pentru a calcula costul sunt folosite toate datele disponibile de antrenare deci ponderile sunt ajustate în funcție de toate exemplele. Pocesul poate fi destul de costistior și ca timp și ca memorie pentru că toate datele trebuie trecute prin rețea, feed-forward, și ponderile sunt ajustate abia la final.
- **Stochastic gradient descent**
- Ponderile sunt ajustate folosind un singur exemplu. Procesul este mult mai rapid însă fluctuează foarte mult.
- **Mini-batch gradient descent**
- pentru optimizarea ponderilor sunt folosite mai multe exemple în setul de antrenare.
- în acest fel parametrii fluctuează mai puțin în procesul de învățare

Iterație vs epocă

- O iterație reprezintă un pas complet de trecere a unui mini-batch prin rețea (inferență/feed forward), calcularea costului și updatarea ponderilor (backpropagation).
- O epocă este finalizată în momentul în care toate mini-batch urile, toate exemplete din setul de date au contribuit la actualizarea ponderilor.
- **După finalizarea unei epoci exemplele de antrenare ar trebui amestecate (random shuffling).**
- Spre exemplu dacă setul de date de antrenare conține 1280 exemple (instanțe, date de antrenare) și mărimea mini-batch ului este de 64, o epocă a conține 20 de iterații.

- Cum variază estimarea erorii/funcției cost în funcție de mărimea mini-batch-ului?
- Cum ar trebui să varieze mini-batch ul în funcție de numărul de clase și de numărul de exemple din ficare clasă?
- Cum ar trebui să fie pasul de actualizare în funcție de mărimea mini-batch-ului?
- Optimizatori (reluați și detaliați) - Curs 6

Bibliografie curs 5,6

- **No free lunch**
  - http://www.no-free-lunch.org/
- **Regresie Logistică**
  - https://towardsdatascience.com/logistic-regression-detailed-overview-46c4da4303bc
  - https://machinelearningmastery.com/logistic-regression-for-machine-learning/
- **Perceptron:** http://130.243.105.49/~lilien/ml/seminars/2007_02_01b-Janecek-Perceptron.pdf

- **Neural Networks**
  - **AND OR perceptron** https://www.youtube.com/watch?time_continue=5&v=Y-ImuxNpS40&feature=emb_logo
- **Neliniarități, funcții de activare:**
  - https://towardsdatascience.com/a-visual-introduction-to-neural-networks-68586b0b733b
  - https://medium.com/the-theory-of-everything/understanding-activation-functions-in-neural-networks-9491262884e0
  - http://colah.github.io/posts/2014-03-NN-Manifolds-Topology/
  - Softmax:
  - https://towardsdatascience.com/the-softmax-function-neural-net-outputs-as-probabilities-and-ensemble-classifiers-9bd94d75932

Bibliografie curs 5,6

- Categorical Cross Entropy
  - https://www.youtube.com/watch?time_continue=69&v=6nUUeQ9AeUA&feature=emb_logo
  - Maximum Likelihood
  - https://www.youtube.com/watch?time_continue=16&v=-xxrisIvD0E&feature=emb_logo
  - https://www.youtube.com/watch?time_continue=4&v=njq6bYrPqSU&feature=emb_logo
  - https://www.youtube.com/watch?time_continue=9&v=iREoPUrpXvE&feature=emb_logo
- Gradienți, chain rule:
  - https://www.youtube.com/watch?v=q555kfIFUCM
  - https://hackernoon.com/dl05-convolutional-neural-networks-1d3bb7fff586
  - https://www.khanacademy.org/math/multivariable-calculus/multivariable-derivatives/gradient-and-directional-derivatives/v/gradient
- Algoritmi de optimizare pe gradient (variante, momentum, RMSProp, Adam)
  - http://ruder.io/optimizing-gradient-descent/
- O parte grafice au fost generate în python folosind matplotlib și paint. Altele din sursele menționate aici